Research: Understanding the Adoption of Metamorphic Testing

Methodology

The study intends to investigate the current state of adoption of metamorphic testing (MT) within the general software community. To achieve this, we intend to conduct two complementary empirical methodologies, that is an online developer survey and large scale project mining.

The developer survey captures the perspectives of software developers by investigating their awareness, previous experience, perceived usefulness and difficulty, and intentions regarding future adoption of metamorphic testing. This methodology provides insight into the human factors influencing adoption and helps explain why developers currently may or may not choose to use metamorphic testing.

The project mining methodology will be done through examining publicly available software repositories hosted on GitHub. Rather than relying on self-reported perceptions, repository mining analyses observable evidence of metamorphic testing adoption through source code, documentation, pull requests, issue discussions, commit messages, and other project artefacts. This allows the study to investigate how metamorphic testing is implemented and discussed within real software projects.

1.0 Survey

1.1 Design

The survey is implemented as an online survey to improve accessibility of developers. Creation and hosting of the survey is done using the online survey platform, Qualtrics.

The survey uses branching logic to dynamically change following questions based on responses to earlier questions. This allows capturing information relevant and specific to participants with different levels of familiarity and experience. Participants who indicate that they have previously used MT are asked more detailed questions about their practical experience, where these questions are not shown to participants who have never applied MT, as they would not be able to provide meaningful responses based on personal experience. Similarly, participants who have not used MT are directed towards questions focused on their perceptions and barriers to adoption. For example, they are asked about their perceived usefulness of MT.

1.1.1 Theoretical Framework

The content of the survey is influenced by the Unified Theory of Acceptance and Use of Technology framework (UTAUT). UTAUT suggests that an individual's behavioural intention (BI) to use a technology is primarily influenced by four core constructs: Performance Expectancy (PE), Effort Expectancy (EE), Social Influence (SI), and Facilitating Conditions (FC).

- Performance Expectancy describes the extent to which an individual believes that using a technology will improve their performance or provide meaningful benefits.
- Effort Expectancy refers to the perceived ease or difficulty associated with learning and using the technology.
- Social Influence captures the impact of others, such as peers, organisations, or communities, on an individual's decision to adopt a technology.
- Facilitating Conditions describe the availability of resources, support, and infrastructure required to enable effective use.

This framework is relevant to the study of MT adoption as it provides a structured approach for investigating why developers may choose to adopt or avoid a testing technique. However, since MT is a specialised software testing methodology rather than a general-purpose technology, the survey extends beyond the original UTAUT as a guideline to construct to capture MT-specific adoption factors. These include developers' awareness and understanding of MT, their ability to identify metamorphic relations, practical experiences applying MT, challenges encountered during implementation, and the availability of domain-specific resources such as tools, documentation, and examples.

1.1.2 Sections

The survey is divided into five main sections: Awareness, Perceptions, Adoption, Challenges, and Intention.

The **Awareness** section investigates participants' existing familiarity with MT, including how they encountered the technique and whether they have accessed relevant learning materials. This provides insight into the current level of exposure and understanding of MT among software practitioners.

The **Perceptions** section examines participants' views of MT, including perceived usefulness, conceptual difficulty, integration difficulty, and potential application areas. These questions primarily relate to Performance Expectancy (PE) and Effort Expectancy (EE) from the UTAUT framework by exploring whether developers believe MT provides value and whether they perceive it as practical to use.

The **Adoption** section is targeted towards participants who have previous experience applying MT in software projects. This section collects information about real-world usage, including the domains where MT has been applied, programming languages and tools used, perceived effectiveness compared to other testing methods, faults identified, and implementation effort. These responses provide practical evidence of current MT adoption and the experiences of developers who have shown actual usage of the technique.

The **Challenges** section focuses on barriers preventing wider adoption of MT, including difficulties with identifying metamorphic relations, lack of resources, insufficient tool support, and integration issues with existing workflows.

The **Intention** section focuses on participants' future willingness to adopt and continue using MT. Participants are asked about their likelihood of using MT in future projects, recommending it to others, and increasing their usage if certain supporting conditions are provided. These questions aim to understand whether positive perceptions of MT translate into potential future adoption, as well as identify factors that may encourage wider adoption. Factors such as improved documentation, clearer use cases, stronger tool support, integration with existing testing workflows, organisational adoption, and evidence of

effectiveness are examined to determine what changes could increase developers' willingness to apply MT.

1.2 Ethics Application

As the survey involves gathering data from human participation, approval of the process is required to ensure ethical guidelines are followed. For this study, a Human Ethics Application has been submitted to the University of Auckland Human Participants Ethics Committee (UAHPEC) describing the full planned process of conducting the survey, including the study description, survey design, data collected and management, target audience, distribution methods, etc.

1.3 Sampling and Distribution

The target audience for the survey are software developers, with no restrictions on experience, knowledge or prior familiarity with metamorphic testing. As long as the developer knows what software testing is and also that they are able to understand and respond to the survey designed in English, they can be considered as eligible. To enable analysis on the broad group, the survey includes a demographic section that collects surveyees' programming experience, employment status and domain, familiar programming languages, etc.

Survey participation is entirely voluntary. Prior to beginning the survey, participants are presented with a Participant Information Sheet explaining the purpose of the study, the types of data collected, confidentiality procedures, and their rights as participants. Responses are collected anonymously through Qualtrics. No personally identifying information is intentionally collected, and participants may discontinue the survey at any time before submission. Survey responses remain anonymous throughout the study and are downloaded only after completion of the data collection period. All data will be stored securely on University-approved storage systems accessible only to the research team.

The online survey will be accessible as a direct link/QR code. The main distribution method will involve social media, where the survey link will be shared in multiple online media, some examples including LinkedIn posts from a software lecturer, on Reddit communities for software developers and testers, Github Discussions relating to testing, etc.

Aside from that, a separate educational website for metamorphic testing (https://metamorphic-testing.vercel.app) has been developed to provide additional online resources that can help developers learn about MT and act as a secondary distribution platform for the survey, which is linked after completion of the website modules. The website is not intended to be a prerequisite for participating in the survey. Instead, it serves as an optional resource designed to provide users with limited or no prior knowledge of MT with a basic understanding of its concepts. By increasing awareness of MT and providing accessible learning materials, the website may help broaden exposure to the survey while ensuring that participation remains open to developers regardless of their existing familiarity with metamorphic testing.

It is expected that the survey will receive around 40+ respondents, however this is an estimated target that will be sufficient to perform significant analysis on.

Project Mining

2.1 Design

GitHub is selected for the source of repository mining due to it being the largest and most popular platform used by developers, with open source software projects across diverse programming languages, domains and communities. This study intends to explore projects with relevance to MT through direct references, implementation identifiers, documentation, discussions, etc.

The mining will be conducted using a custom script developed in Python due to its strong support for data manipulation and simplicity. The script's process consists of four main phases:

1. Search: Identify repositories that may contain MT through broad GitHub searches
2. Clone and Filter: Prepare identified repositories for analysis by retrieving necessary files to local environment
3. Local Analysis: Analysis on local environment to detect adoption of MT within retrieved contents for simple classifications.
4. Visualisation: Generate statistical summaries and visual representations of MT adoption patterns.

2.2 Mining Pipeline

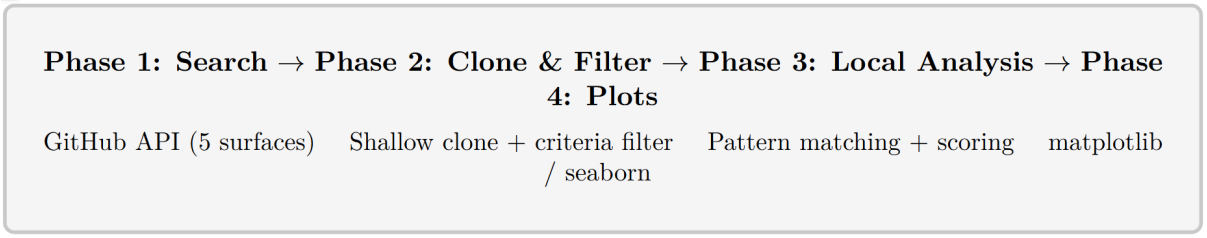


Figure: Overview of the four-phase repository mining pipeline.

Phase 1: Search

The first phase performs searches across multiple Github surfaces to capture five types of different data sources in projects

- Repository metadata, including names, descriptions, topics, and README files
- Source code files
- Commit messages
- Issues and pull requests
- GitHub Discussions

The search strategy uses a broad collection of MT-related keywords, including "metamorphic testing", "metamorphic test", "metamorphic relation", "metamorphic relations", "metamorphic tests", etc additional variations. The searching phase is broad, indicating that MT related terminology has appeared somewhere, but not whether it has been actually implemented. This is intentional as the analysis stage later on is intended to perform actual verification.

To search through projects, Github's REST API and GraphQL (for discussions) are used, where there exist limitations, including a maximum result count per query. To improve coverage, adaptive date-range slicing is applied when queries return excessive results. Large search ranges are recursively divided into smaller time windows until the number of results falls within manageable limits. Results obtained across multiple queries and time windows are deduplicated to produce a final repository candidate dataset.

The output of this stage includes repository metadata and references to relevant artefacts for subsequent filtering and analysis.

Phase 2: Clone and Filter

The second phase prepares the references gathered from the search phase for local analysis. Due to the broad search criteria in the first phase, custom filtering is further applied to ensure repositories chosen for the extensive analysis process are high quality and can represent "general" software development. These filters include whether the repository is a fork without significant independent development, the size of the repository as that may affect feasibility of analysis, repository activity and availability, and an additional relevance criteria check to MT.

Repositories satisfying these criteria are shallow-cloned locally using Git. Shallow cloning retrieves only the latest repository state while reducing storage and processing requirements. Since the analysis focuses on current source files, documentation, and repository artefacts, complete commit history is unnecessary.

A repository manifest is maintained to record repository metadata, clone status, filtering outcomes and analysis results. This ensures that the mining process remains reproducible and traceable.

Phase 3: Local Analysis and Classification

The third phase performs detailed analysis on repository files cloned locally from phase 2. The analysis pipeline scans repository files and applies pattern-based detection methods using regex. Detection patterns are grouped into several categories as shown in the table below.

Pattern Category	Examples
Library imports / identifiers	*import metamorphic_library … from metamorphic…, MetamorphicTest, Metamorphic Relation, @metamorphic_label* (classes, functions, methods, libraries, etc in code)
Documentation, comments and strings	*# metamorphic testing, // metamorphic relations, string = "metamorphic relation"* (code comments, documentation strings, README.md, configuration files, string literals, etc)
Lemmatised patterns	Morphological variants of MT not included in previous exact search strings, e.g. "metamorphically").

	(Matched against exact comment/docstrings using Python's NLTK Lemmatizer)
Language-specific conventions	Java method names matching *testMR()* (following MT literature naming convention; Java class name *MetamorphicTesting*

Based on the detected patterns as above, a score is computed for each analysed repository, allowing for classification of the project into tiers based on the amount of patterns. This classification allows distinguishing between simple keyword presence and actual evidence of MT adoption.

Strong Adoption

Repositories containing multiple explicit, independent MT indicators, such as implementation references combined with documentation or comments. These repositories provide strong evidence of deliberate MT usage.

Weak Adoption

Repositories containing limited MT evidence, such as isolated references or small experimental implementations. These repositories indicate MT awareness but provide weaker evidence of sustained adoption.

Limited Adoption

Repositories identified during the search stage but where no explicit MT evidence is detected during local analysis. These repositories may contain MT terminology in external references, discussions, or unrelated contexts.

Phase 4: Visualisation

The final phase generates visual summaries of the repository mining results. These visualisations provide an overview of MT adoption trends and characteristics across the analysed repository population. The generated analyses include

- Distribution of repositories by MT adoption classification
- Programming languages across repositories
- Repository creation trends over time
- Relationship between repository popularity and MT evidence
- Frequency of detected MT patterns
- Distribution of search sources and overlap between queries

These visualisations support interpretation of our research question (How prevalent is explicit metamorphic testing adoption across large-online resources, and what project characteristics are associated with its presence or absence?) by highlighting where explicit MT adoption occurs, which technologies are associated with adoption, and how frequently MT appears across open-source projects.